

# Resume



안녕하세요 이영민입니다.

## Email

[bbok4yo@gmail.com](mailto:bbok4yo@gmail.com)

## Channel

[Github](#)

[Velog](#)

## ABOUT ME

- React/Next.js (App Router) 기반 프로젝트에서 동적 라우팅 및 전역 상태 관리를 포함한 아키텍처를 설계하고 구축합니다.
- Git/PR 기반 협업에 익숙하며, 기획 의도를 명확히 파악하는 세밀한 커뮤니케이션을 주도합니다.
- 컴포넌트 재사용성과 API 호출 타이밍 등 실제 서비스 관점에서 구조 설계를 주도하며 성능 최적화를 추진합니다.
- 함께 일하고 싶은 동료로서, 기술과 더불어 좋은 팀 문화를 만드는 데 적극 기여합니다.
- 백엔드 로직과 서버 환경에 대한 이해를 바탕으로 데이터 구조를 함께 설계하며, 효율적인 API 연동을 주도합니다.
- TypeScript 도입 및 성능 최적화를 통해 코드의 안정성을 확보하고, 사용자에게 쾌적한 디지털 경험을 제공합니다.
- AI와 협업하여 요구사항 분석부터 SDD 설계, 자동화된 코드 품질 검증까지 소프트웨어 엔지니어링 전 과정을 최적화합니다.

## CAREERS

### 리얼라인

기간: 2026.01 ~ 2026.02

- AI 기반 소프트웨어 엔지니어링 파이프라인 구축: 단순 코드 생성을 넘어 요구사항 분석, SDD 설계, 구현·품질 검증 전 과정에 Claude Code를 통합하여 개발 주기를 단축했습니다.
- 단기 다중 프로젝트 완수: 1개월 내 전략 보고서 생성기·지도 데이터 시각화·스팟업 웹 3종을 스펙 기반 AI 워크플로우로 구현했습니다.

성과: AI를 활용한 일관된 개발 및 품질 엔지니어링

#### 1. AI-Assisted SDD 개발 및 문서화

- 아키텍처 설계: 요구사항을 바탕으로 Claude Code와 상호작용하며 시스템 아키텍처, 데이터 모델링, 컴포넌트 계층을 정의하는 SDD를 신속하게 작성했습니다.
- 문서화 자동화: API 명세 변경, 상태 관리 플로우 등을 AI를 통해 실시간으로 반영하여 프로젝트 가시성과 협업 효율성을 확보했습니다.

#### 2. Custom Skill-Hook 기반 코드 품질 엔지니어링

- **자동화 품질 검증:** Custom Skill과 Git Hook(Pre-commit/Pre-push)을 연동하여 커밋 단계에서 AI가 정적 분석·컨벤션 검사를 자동 수행하도록 구성했습니다.
- **지속적 리팩토링:** AI가 프로젝트 전체 컨텍스트를 파악하여 보안 취약점 점검, 중복 코드 제거, 아키텍처 개선안을 선제적으로 제안하고 적용했습니다.

## 라운피플

기간: 2024.01 ~ 2024.04

성과: 테스트 케이스(TC) 및 주어진 테스트 항목에 기반한 기능 테스트 수행

- JIRA를 활용해 버그 리포트를 생성하고 이슈를 트래킹했습니다.
- 테스트 결과 보고서를 작성하고 개발팀에 피드백을 전달했습니다.
- 관련 기술: JIRA

## ACTIVITIES

---

### 프론트엔드 개발자 과정

소속/기관: Kakao x goorm

활동 연도: 2024.07~2025.04

- Next.js를 학습하고 실제 프로젝트에 적용해 실전 감각을 키웠습니다.
- 디자이너, PM, 백엔드 개발자 등 다양한 직군과 협업하면서 직군별 의사결정 우선순위 차이를 좁히는 커뮤니케이션 방식을 익혔습니다.

### 캠퍼스 SW 아카데미 TABA 빅데이터 분석 및 인공지능 처리를 위한 SW융합개발자양성

소속/기관: 과학기술정보통신부

활동 연도: 2023

- 프론트엔드·백엔드 개발 및 Figma 기반의 기획 등 전반적인 개발 협업 과정을 경험했습니다.
- 협업 톨과 개발 전반을 아우르며 실제 프로젝트에서의 기본적인 협업 프로세스를 이해했습니다.
- 과정 수료 후 프로젝트 부분에서 우수상 및 우수교육생으로 선정되었습니다.

## EDUCATION

---

- 단국대학교 컴퓨터공학과 : 2019.03. ~ (졸업예정)

## STACKS

---

### Front-End

- HTML, CSS(SCSS), JS(ES6) & TS
- React.js, Next.js
- tailwind
- Redux Toolkit, react-query, zustand

- D3.js

## Collaboration & Tools

- slack
- Figma
- Git, Github

## PROJECTS

---

### MindGraph (AI 지식 캡처 & 그래프 시각화)

서비스 개요: ChatGPT·Gemini·Claude·Grok의 답변을 캡처 즉시 의미 단위로 묶어 D3.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. 기획·설계·개발·QA 전 구간을 1인 PO로 수행했고 도메인 getmindgraph.com 등록 후 출시 전 1인 프로토타입 단계입니다.

팀원: 1명 (1인 PO·풀스택 개발)

기술 스택: TypeScript, Next.js 16 App Router, React 19, D3.js, Chrome Extension Manifest V3, IndexedDB, Supabase, Claude Code, GitHub Actions

- 1인 PO 환경에서 가설을 빠르게 검증하기 위해 Claude Code 위에 9개 혹·plan·qa·ship 3 phase /sprint·4중 SSOT 자동 동기를 설계해 sprint 라이프사이클 반복 작업을 자동화했습니다.
- 매 prompt에 전체 docs가 끌려오는 컨텍스트 과부하를 줄이기 위해 7부서·9에이전트별 docs를 분리하고 task\_type·코드 path glob을 must-read doc에 매핑해 첨부량을 5개 내외로 줄였습니다.
- 출시 후 가설 검증 인프라를 사전 구축하기 위해 PostHog·Sentry·OpenTelemetry·api/feedback·api/error-log 5개 채널을 코드 베이스에 모았습니다.
- 추천 정확도와 클릭률을 출시 전에 측정하기 위해 lib/telemetry.ts에 AccuracyEvent(top-3 ≥ 60%)·InteractionEvent(클릭률 ≥ 60%)와 nodeId 한정 적재 정책을 함께 설계했습니다.
- 네트워크 불안정 환경의 캡처 유실을 막기 위해 IndexedDB 우선 기록과 Pending Ops 큐(MAX\_RETRIES=5) 기반 Write-Behind 동기화로 오프라인 CRUD와 온라인 복귀 자동 flush를 구현했습니다.
- TopicNode·QANode 분리 구조의 교차 조회 비용을 해결하기 위해 nodeType·parentId·path·childIds·depth 메타데이터를 가진 UnifiedNode 단일 모델을 설계해 조상 경로 조회를 path 한 번 분해로 처리했습니다.
- Supabase RLS 정책 직접 설계와 GitHub Actions CI/CD·next-intl 한·영 다국어·MV3와 Web origin 화이트리스트 검증 까지 1인이 운영했습니다.

### chatGraph (AI 대화 시각화 플랫폼)

서비스 개요: LLM과의 대화를 꼬리물기 형태의 마인드맵 그래프로 시각화해 비선형 사고 흐름을 보존하는 웹 서비스. 4인 팀의 FE 한 명으로 응답 대기 동안의 사용자 흐름과 시각화 라이프사이클을 담당했습니다.

팀원: 4명 (FE 2명, BE 2명)

기술 스택: TypeScript, React, Next.js, D3.js, Zustand, Tailwind CSS, TanStack Query, Vitest

- 새 토픽 생성 시 LLM 응답 대기로 화면이 멈추는 문제를 해결하기 위해 sessionStorage에 임시 프롬프트를 저장하고 temp ID로 즉시 라우팅한 뒤 mutation 성공 시점에 실제 ID로 교체하는 Optimistic 라우팅을 설계해 입력 직후 곧바로 채팅 화면을 볼 수 있게 했습니다.
- 사이드바 토픽 클릭 시 초기 로딩 지연을 줄이기 위해 Zustand 스토어에 응답을 프리패치해 두고 useSuspenseQuery의 initialData로 주입하는 패턴을 적용해 라우트 전환 시 빈 화면 노출 없이 데이터가 즉시 그려지도록 구현했습니다.

- 단일 파일에 843줄까지 비대해진 use-question-tree 혹은 책임 단위로 분리해 355줄로 축소했고 Feature-First 구조로 도메인·조립·공통을 분리해 확장성을 확보했습니다.
- 거대해진 페이지 컴포넌트의 책임을 분산하기 위해 TopicContentLayout·StandardChatView·OptimisticChatView로 뷰 계층을 나누고 findPathToNode 경로 탐색·Zustand 토픽 토글·새 토픽 시작 혹은 Vitest 회귀 테스트를 작성해 핵심 사용자 흐름의 회귀를 자동 감지하도록 했습니다.
- 팀원이 도입한 D3.js Force Simulation 시각화가 React 리렌더 시 SVG 트리와 충돌하는 문제를 해결하기 위해 useEffect cleanup과 `d3.selectAll("*").remove()` 패턴을 결합한 라이프사이클 통합 코드를 담당했습니다.

## Fit (사일런트 컨퍼런스 플랫폼)

서비스 개요: 무선 헤드폰을 통해 다중 채널의 발표를 골라 듣는 사일런트 컨퍼런스의 온라인·오프라인 하이브리드 중계 플랫폼

팀원: 5명 (FE 3명, BE 2명)

기술 스택: Vite, React, WebRTC, STOMP, SockJS, Framer Motion, Redux Toolkit, redux-persist, Tailwind CSS

- 현장 청취를 위한 수백 밀리초 단위 저지연 오디오 전송이 필요해 RTCPeerConnection API로 발표자·청중 양측 시그널링 클라이언트를 구현하고 STOMP 토픽 위에서 SDP-ICE 후보를 교환하는 1:N 오디오 송수신 흐름을 설계했습니다.
- SDP 협상이 끝나기 전 도착한 ICE 후보가 유실되어 청중이 발표자 음성에 접속하지 못하는 문제를 해결하기 위해 컴포넌트 ref에 ICE 큐를 두고 setRemoteDescription 직후 일괄 flush하는 버퍼링 로직을 적용해 PeerConnection 성공률을 끌어 올렸습니다.
- 모바일 망과 행사장 Wi-Fi의 일시적 단절에 대비해 STOMP 클라이언트에 reconnectDelay 5초와 4초 간격 양방향 heartbeat를 설정하고 자체 운영 TURN 서버를 ICE 후보군에 추가해 NAT 환경에서도 청취가 끊기지 않도록 했습니다.
- 시그널링 트래픽과 혼잡도 데이터가 서로의 채널을 점유하지 않도록 혼잡도 전용 STOMP 클라이언트를 별도로 구성해 `/sub/session` 토픽만 구독하게 분리했고 채널 간 인원 쏠림을 실시간 화면에 반영했습니다.
- 인증 토큰과 좋아요 세션 목록을 Redux Toolkit으로 관리하고 redux-persist whitelist에 auth만 포함해 새로고침 후에도 세션 컨텍스트를 유지하면서 휘발성 데이터가 영속화되는 부작용을 막았습니다.

## xAB (A/B 테스트 기반 SNS)

서비스 개요: 두 가지 선택지에 대한 실시간 투표 및 댓글 토론이 가능한 소셜 네트워크 서비스. Next.js App Router 위에서 인증·실시간 동기화·모달 라우팅을 풀스택으로 담당했습니다.

팀원: 4명 (FE 4명)

기술 스택: TypeScript, Next.js, React Query, Zustand, Supabase, Tailwind CSS

- 게시물 상세 진입 시 피드 위치와 맥락이 끊기는 문제를 해결하기 위해 Next.js App Router의 병렬 라우트와 인터셉팅 라우트를 결합한 모달 라우팅을 구현해 피드 위치를 유지한 상태에서 상세 화면을 띄우는 탐색 경험을 제공했습니다.
- 새로고침 없이 다른 사용자의 댓글·투표가 즉시 반영되도록 Supabase Realtime의 `postgres_changes` 이벤트 구독을 도입하고 React Query 캐시 갱신 로직과 연동해 실시간 동기화 환경을 구축했습니다.
- 인증되지 않은 사용자의 보호 경로 접근을 차단하기 위해 `@supabase/ssr` 기반 Next.js 미들웨어를 도입해 매 요청마다 서버 측에서 세션을 검증하고 미인증 사용자를 로그인 페이지로 리다이렉트하도록 구성했습니다.
- 메인 피드 초기 로딩 지연을 줄이기 위해 TanStack Query의 `useInfiniteQuery` 와 Intersection Observer 트리거를 결합해 페이지 단위 분할 호출과 지연 로딩이 적용된 무한 스크롤 피드를 구현했습니다.
- 로드밸런서 환경에서 OAuth 콜백 오리진이 잘못 구성되어 로그인 직후 사용자가 잘못된 도메인으로 떨어지는 문제를 방지하기 위해 콜백 라우트에서 `X-Forwarded-Host` 헤더를 우선 참조해 원본 오리진을 복원하는 분기를 구현했습니다.

